

Tutorial 8: WRES1201 – Computer System Architecture

1. Explain the function of the EU (execution unit) and the BIU (bus interface unit).
 - **BIU (Bus Interface Unit):** Handles all communication with memory and I/O over the system bus. It fetches instructions ahead of time into an **instruction prefetch queue**, fetches operands, and writes back results. It is the unit that interfaces the processor with the outside world.
 - **EU (Execution Unit):** Takes instructions from the prefetch queue, decodes them, and executes them (performs ALU operations, manipulates registers).
2. Why does pipelining improve performance?
 - Pipelining divides instruction processing into several stages (e.g., fetch, decode, execute). Instead of completing one instruction fully before starting the next, multiple instructions are **overlapped**. Each stage works on a different instruction at the same time, like an assembly line. This increases instruction **throughput** (instructions completed per unit time) without reducing the time for any single instruction. With k stages, the pipeline can in principle approach a k -fold speedup over sequential execution.
3. What is pipeline latency?
 - **Pipeline latency** is the time it takes for a *single* instruction to pass through the *entire* pipeline (from entering the first stage to leaving the last stage). Pipelining improves throughput, not latency. The latency of one instruction may actually be slightly higher than unpipelined (due to latch/stage register overhead), even though many instructions complete faster overall.

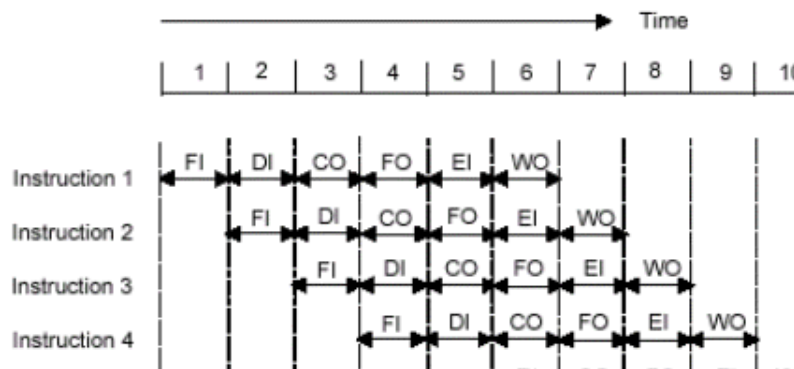


Figure 1

4. Consider the timing diagram of Figure 1. Assume that there is only a two stage pipeline (fetch, execute). Redraw the diagram to show how many time units are now needed for four instruction. What is *speedup 2-stage pipeline* comparing with sequential pipelined.
5. How many time units are needed to execute 600 instructions with 6-stage pipeline. And how many time units if unpipelined. . Calculate the *speedup*.
6. Given an unpipelined processor with 10ns cycle time and pipeline latches with 0.5ns latency, what are the cycle times of pipelined versions of the processor with 2, 4, 8, and 16 stages if the datapath logic is evenly divided among the pipeline stages? Also, what is the latency of each of the pipelined versions of processor?
7. List and briefly explain various ways in which an instruction pipeline can deal with conditional branch instruction.
 - **Multiple streams:** Replicate the early pipeline stages and fetch *both* the branch-taken and branch-not-taken paths, choosing the correct one once the branch resolves. Costly (contention, extra hardware) and doesn't scale to multiple branches.
 - **Prefetch branch target:** When a conditional branch is recognized, the target instruction is prefetched (in addition to the next sequential instruction) so it's ready if the branch is taken.
 - **Loop buffer:** A small, very fast memory holding the n most recently fetched instructions. Useful for loops and short forward branches — if the target is in the buffer, no memory fetch is needed.
 - **Branch prediction:** Predict whether a branch will be taken and fetch accordingly. Methods include:
 - *Static:* predict-never-taken, predict-always-taken, or predict by opcode.
 - *Dynamic:* taken/not-taken switch bits or a branch history table updated by recent behavior.
 - **Delayed branch:** Reorder instructions (by the compiler) so that an instruction that must execute regardless is placed *after* the branch (in the branch delay slot), making use of the cycle that would otherwise be wasted.

4.	Time :	1	2	3	4	5
	Instruction 1 :	[F]	[E]			
	Instruction 2 :		[F]	[E]		
	Instruction 3 :			[F]	[E]	
	Instruction 4 :				[F]	[E]

Parallel: Time : 5 time units / Formula, $time = k + (n-1)$
 $= 0 + (4-1)$
 $= 5 \text{ units}$

Sequential: $2 \text{ unit} \times 4 = 8 \text{ units}$

$$\text{Speedup} = \frac{8}{5}$$

$$= 1.6$$

5. Pipelined: $time = 6 + (600-1)$
 $= 605 \text{ time units}$

$$\text{Speedup} = \frac{3600}{605}$$

$$= 5.95$$

Unpipelined: $6 \times 600 = 3600 \text{ time units}$

6. Unpipelined cycle time = 10ns
 Latch latency = 0.5ns per stage

2 stages

$$\text{Cycle time} = 10\text{ns} \div 2 + 0.5$$

$$= 5.5\text{ns}$$

$$\text{Latency} = 5.5 \times 2$$

$$= 11.0\text{ns}$$

4 stages

$$\text{Cycle time} = 10\text{ns} \div 4 + 0.5$$

$$= 3.0\text{ns}$$

$$\text{Latency} = 3.0 \times 4$$

$$= 12.0\text{ns}$$

8 stages

$$\text{Cycle time} = 10\text{ns} \div 8 + 0.5$$

$$= 1.75\text{ns}$$

$$\text{Latency} = 1.75 \times 8$$

$$= 14.0\text{ns}$$

16 stages

$$\text{Cycle time} = 10\text{ns} \div 16 + 0.5$$

$$= 1.125\text{ns}$$

$$\text{Latency} = 1.125 \times 16$$

$$= 18.0\text{ns}$$

$\therefore$ The more stages, the shorter the cycle time, the higher latency for single instruction.